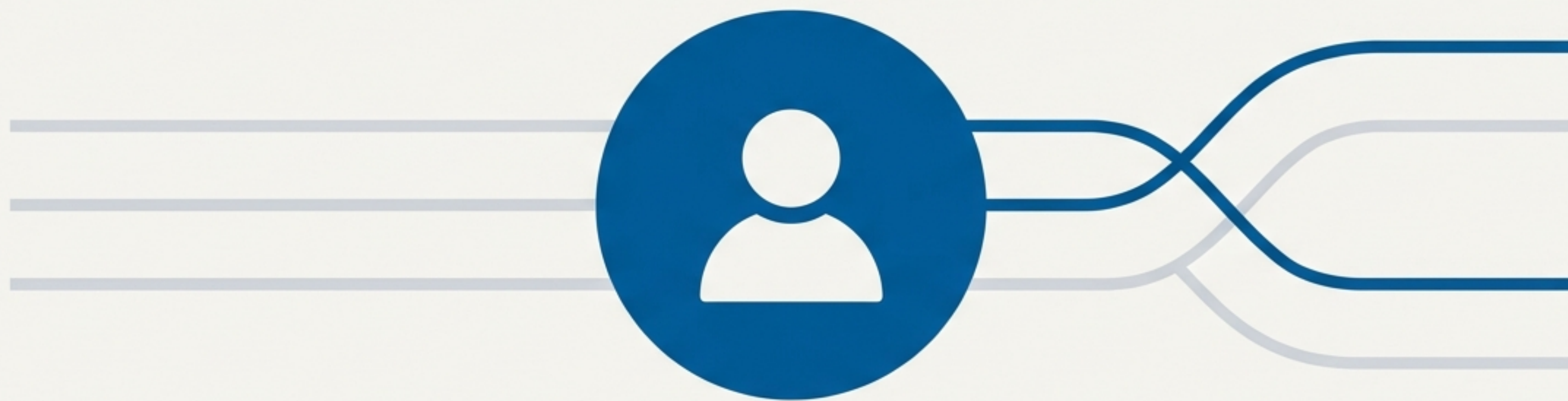


LangGraph workflows and human-machine interaction

Driving large models: from automation to human-machine collaboration



本次分享大纲

1. 问题的提出：大语言模型应用中的“信任悖论”
 2. 核心理念：引入“人机回环”实现协同智能
 3. 关键机制：工作流的“暂停”与“记忆” (Interrupt' & 'Checkpoint')
- ```
Interpointer_Σ0E s = new _miturns_teE() (`Interrupt` & `Checkpoint`)
```
4. 实战演练：构建一个可中断和恢复的工作流
  5. 总结：开启人机协同新范式



# LLM应用中的“信任悖论”

我们如何信任一个我们无法完全控制的系统？

承诺：强大的自主能力



内容创作



数据分析



工具使用

风险：不可预测的结果



事实错误（幻觉）

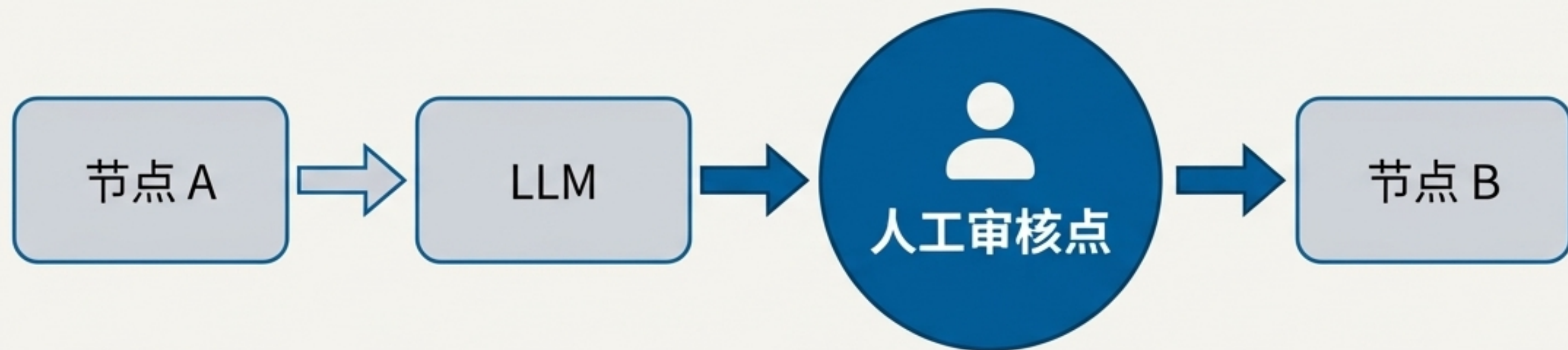


合规风险



不当输出

## 解决方案：引入“人机回环”（Human-in-the-Loop）



“人机回环”将人类的判断力、审查和决策无缝集成到自动化工作流程中，确保关键步骤的可靠性。

✓ **提升可靠性：**在关键步骤进行事实核查与修正。

✓ **确保合规性：**人工审核敏感操作与输出。

✓ **增强灵活性：**动态调整 workflow 路径。



# LangGraph中的典型人机交互场景



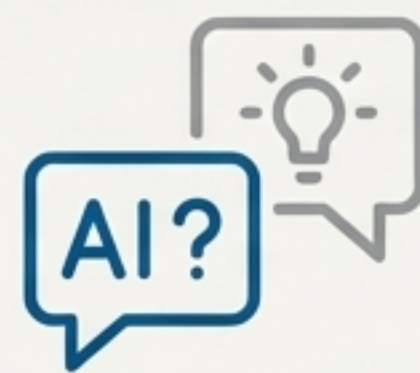
## 审查工具调用

AI: 准备调用支付API...  
→ 人工: **[批准/拒绝]** →  
AI: API调用已执行/已取消。



## 验证LLM输出

AI: 生成了一篇报告草稿...  
→ 人工: **[编辑并确认]** →  
AI: 报告最终版已确认。



## 提供额外上下文

AI: 我需要更多信息才能继续...  
→ 人工: **[输入补充信息]** →  
AI: 收到, 继续执行。



# 关键机制 1: 工作流的“暂停键” - `Interrupt`

- `Interrupt` 是一个特殊的返回值。
- 当节点返回它时，会安全地暂停图（Graph）的执行，等待外部（人类）的输入。

导入 `Interrupt`

```
从 langgraph.types 导入 Interrupt
from langgraph.types import Interrupt
```

返回它以暂停

```
def human_approval_node(state):
 # ... 业务逻辑 ...
 print("等待人工审核...")
 # 返回 Interrupt() 对象以暂停执行
 return Interrupt()
```





## 关键机制 2：工作流的“存档点” - `Checkpoint`

**核心问题：** 工作流暂停了，但它的状态和历史记录怎么办？

**解决方案：** `Checkpoint` 负责持久化（persistence）。它将每一步的状态保存下来，使得工作流可以随时从中断点恢复。

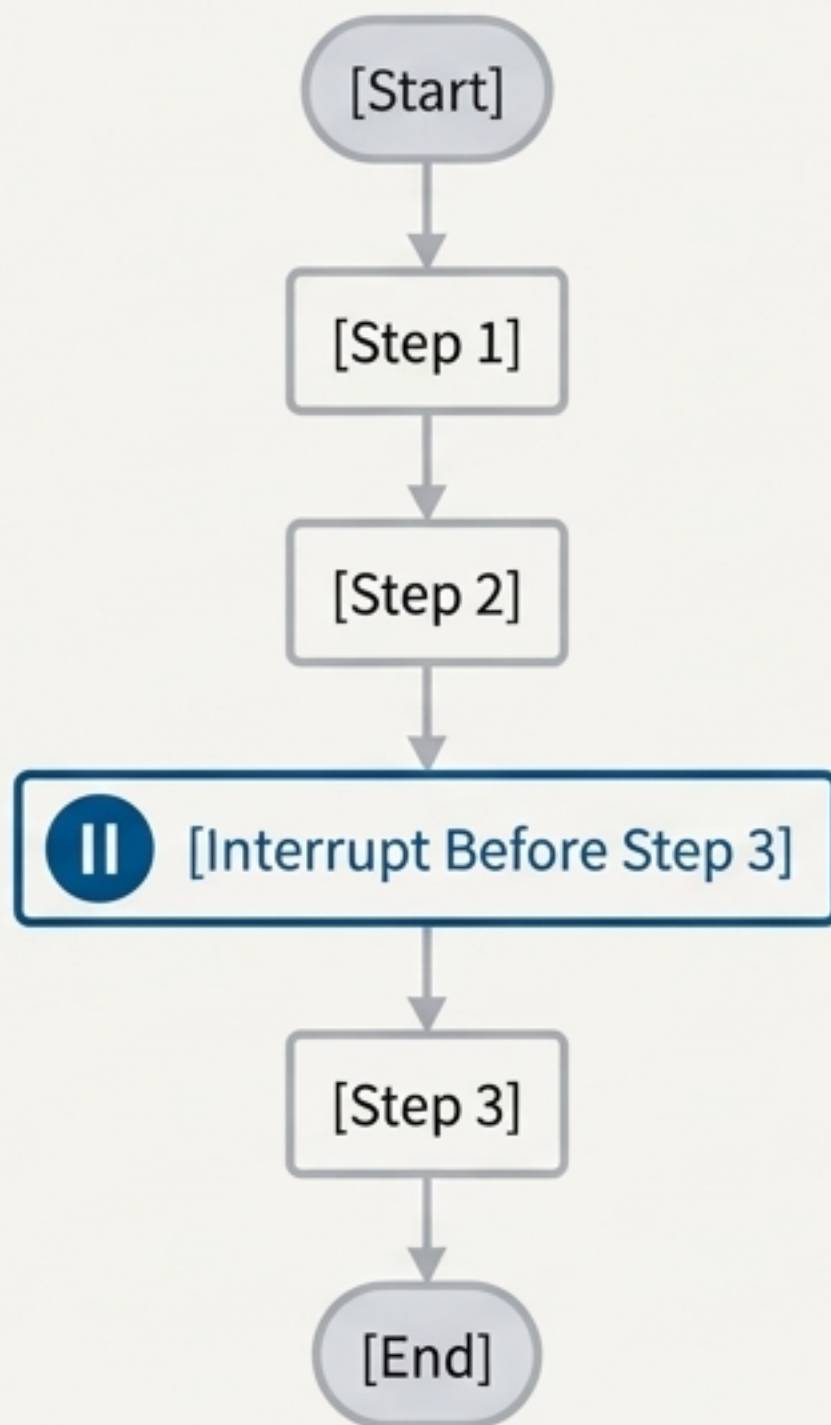


**关键概念：** `thread\_id` 确保了每个会话的状态是独立且安全的。每个用户的交互都在自己的“存档”中进行。



# 实战演练：构建带有人工审核节点的流程

**\*\*目标\*\***：我们将构建一个三步流程。在进入第三步（`step\_3`）之前，流程将暂停，等待用户确认后才能继续。





# 演练 1/4: 定义状态与节点

```
from typing import TypedDict
from langgraph.graph import StateGraph
```

# 1. 定义状态

```
class State(TypedDict):
 input: str
```

# 2. 定义节点 (简单的打印函数)

```
def step_1(state: State) -> State:
 print("--- 执行步骤 1 ---")
 return {"input": state["input"]}
```

```
def step_2(state: State) -> State:
 print("--- 执行步骤 2 ---")
 return {"input": state["input"]}
```

```
def step_3(state: State) -> State:
 print("--- 执行步骤 3 ---")
 return {"input": state["input"]}
```

```
def step_3(state: State) -> State:
 print("--- 执行步骤 3 ---")
 return {"input": state["input"]}
```

# 3. 构建图骨架

```
builder = StateGraph(State)
builder.add_node("step_1", step_1)
builder.add_node("step_2", step_2)
builder.add_node("step_3", step_3)
builder.set_entry_point("step_1")
builder.add_edge("step_1", "step_2")
builder.add_edge("step_2", "step_3")
builder.add_edge("step_2", "step_3")
builder.add_edge("step_3", "__end__")
```

这是任何LangGraph应用的基础结构：状态、节点和边。



## 演练 2/4：设置中断点

```
from langgraph.checkpoint.memory import MemorySaver

设置一个内存Checkpoint来保存状态
memory = MemorySaver()

编译图，并设置中断点
graph = builder.compile(
 checkpointer=memory,
 # 在执行 step_3 节点前中断
 interrupt_before=["step_3"]
)
```

核心代码：在这里声明我们的“暂停”规则。



## 演练 3/4：运行与中断

```
配置线程ID, 用于状态管理
config = {"configurable": {"thread_id": "user_1"}}

首次运行
print("首次运行图...")
for event in graph.stream({"input": "Hello"}, config):
 # 打印事件流
 pass # 在实际应用中处理事件

print("\n图的执行已暂停。")
```

首次运行图...

--- 执行步骤 1 ---

--- 执行步骤 2 ---

图的执行已暂停。

注意：流程在打印“执行步骤 2”后停止，并未进入“步骤 3”。



## 演练 4/4: 人工干预与恢复

```
模拟人工输入
user_approval = input("是否继续执行步骤 3? (yes/no): ")

if user_approval.lower() == "yes":
 print("\n正在恢复执行...")
 # 传入 None 来从中断点恢复
 for event in graph.stream(None, config):
 pass
else:
 print("\n操作已取消。")

是否继续执行步骤 3? (yes/no): yes

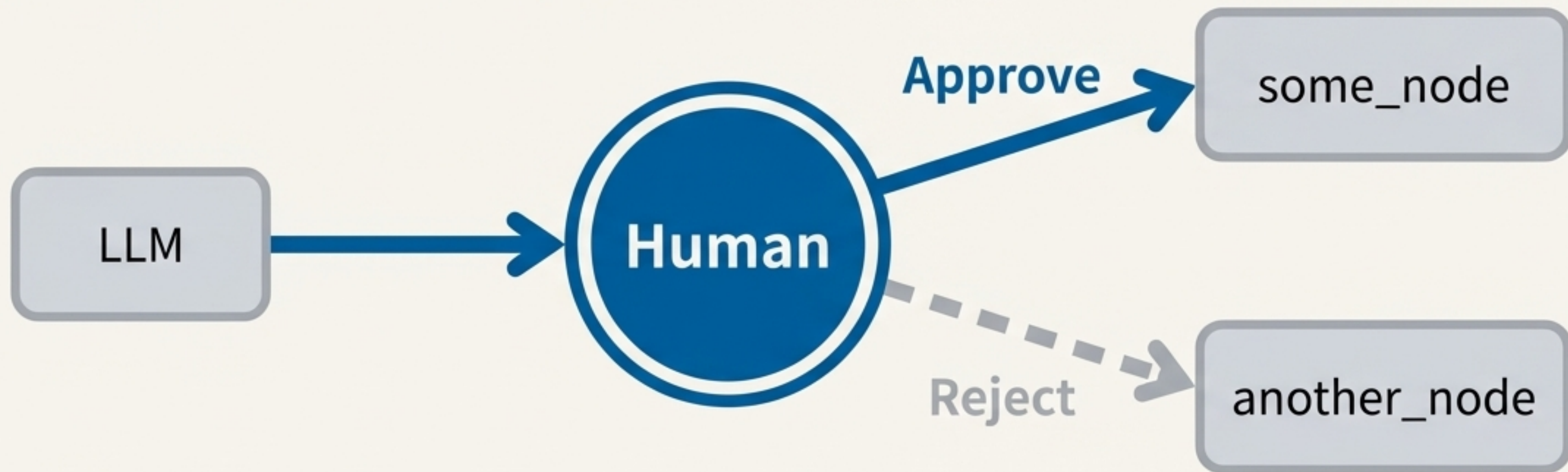
正在恢复执行...
--- 执行步骤 3 ---
```

关键点: 传入 `None` 告诉  
LangGraph从上一个检查点继续。



# 进阶模式：基于人工反馈的动态路由

**核心理念：**中断点不仅可以暂停，更可以成为决策的分叉路口。根据人类的输入，工作流可以动态地流向不同的分支，实现真正的灵活协同。



这种模式对于批准/拒绝工作流（如API调用前审核）或需要根据用户反馈选择下一步操作的场景至关重要。

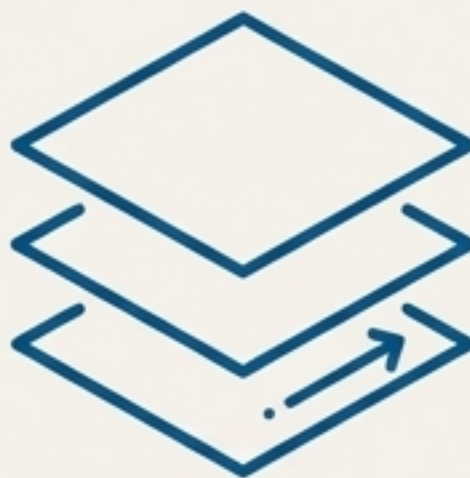


# 总结：LangGraph 人机协同的核心



## [控制 Control]

通过 `Interrupt` 或 `interrupt\_before` 在任意节点设置“暂停点”，将控制权交还给人类。



## [记忆 Memory]

`Checkpoint` 负责状态持久化，确保中断和恢复无缝衔接，让每一次交互都有迹可循。



## [协同 Collaboration]

将人类的审查、决策和智慧融入AI工作流，构建更强大、更可靠、更值得信赖的智能系统。

*LangGraph提供的工具，帮助我们超越简单的自动化，构建真正智能的人机协同应用。*